DR. D.Y. PATIL PRATISHTHAN ENGINEERING COLLEGE

Salokhenagar, Kolhapur

A PROJECT REPORT

ON

HouseHunt

HOUSE RENTAL MANAGEMENT SYSTEM

A full-stack web application to simplify the process of finding, listing, and managing rental properties.



Submitted in partial fulfillment of the requirements for SmartBridge Internship Program

Submitted By

Vaishnavi Rajshekhar Gaikwad

B.Tech (Computer Science and Engineering)

Roll No.: 125

Under the Guidance of

SmartBridge Team

Academic Year

2025 – 2026

INTRODUCTION

HouseHunt is a user-centric web application designed to simplify the process of discovering and managing residential rental properties. It provides a unified digital platform for prospective tenants to browse available homes, compare relevant property details, and connect with property owners or listing representatives. The application aims to reduce the effort involved in traditional house-hunting by organizing property information in an accessible interface.

The proposed experience emphasizes property listings, location-based discovery, budget and amenity filtering, property details, and contact or enquiry workflows. HouseHunt is documented as a MERN full-stack project, combining React for the user interface with a Node.js and Express.js backend and MongoDB for persistent application data. The deployed frontend is available at https://househunt-smartbridge.netlify.app/.

SCENARIO BASED CASE STUDY

Background

Students, working professionals, and families frequently spend considerable time searching for suitable accommodation. Listings are often scattered across different sources, making it difficult to compare price, location, amenities, and availability. HouseHunt addresses this problem through a centralized rental-property discovery experience.

Scenario and User Persona

- Name: Riya Sharma (illustrative persona)
- Age: 25 years
- Profession: Software Engineer
- Goal: Find a furnished rental home close to the workplace within a fixed monthly budget.
- Challenges: Limited time for visits, inconsistent listing information, and difficulty comparing multiple properties.

The Challenge

Riya needs a straightforward way to narrow rental choices by city or area, price, property type, and facilities. She also needs clear listing details and a reliable way to express interest in a suitable property.

Solution: How HouseHunt Helps

Riya opens HouseHunt, browses property cards, applies available search filters, and reviews the details of shortlisted homes. Where the implementation supports enquiries, she can contact the owner or agent. This workflow makes initial property discovery more organized and reduces unnecessary manual comparisons. This scenario is illustrative and is not a measured user outcome.

SYSTEM REQUIREMENTS

Software Requirements

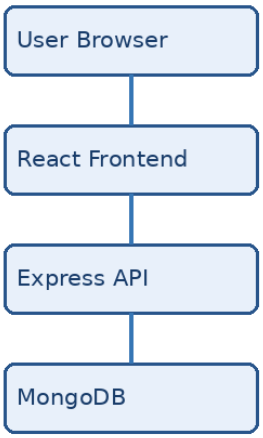
- Operating System: Windows 10/11, macOS, or Linux.
- Node.js and npm: JavaScript runtime and package management.

- React.js: Frontend interface development.
- Express.js and Node.js: Backend REST API development.
- MongoDB and Mongoose: Database storage and schema modeling.
- Visual Studio Code: Source-code editing and debugging.
- Postman or equivalent: API testing.
- Modern browser: Chrome, Firefox, or Edge.
- Git and GitHub: Source control; Netlify: frontend hosting.

Hardware Requirements

- Processor: Intel Core i3/i5 or equivalent.
- RAM: 8 GB minimum; 16 GB recommended for development.
- Storage: At least 2 GB available for tools and project dependencies.
- Internet: Required for deployment, package installation, and hosted services.

TECHNICAL ARCHITECTURE

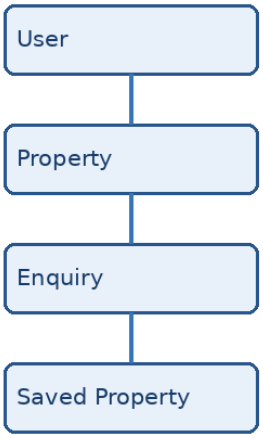


The proposed architecture follows a client-server design. The React client renders the property browsing experience and sends HTTP requests to the Express API. The server validates requests, applies business rules, and accesses MongoDB through Mongoose models. Responses return to the frontend for presentation. Authentication and authorization should be enforced on protected operations such as managing listings.

Architecture Components

- User Interface: Home page, property listing cards, search/filter controls, property detail views, and forms.
- API Layer: Routes for users, properties, and enquiries where implemented.
- Business Logic: Validates inputs and manages property or user workflows.
- Database: Stores user profiles, listing details, and associated records.
- Hosting: Public frontend deployment on Netlify; backend hosting depends on the actual deployment configuration.

ER DIAGRAM



Conceptual entities (verify against implemented MongoDB collections): User (userId, name, email, passwordHash, role); Property (propertyId, ownerId, title, location, rent, type, amenities, images, availability); Enquiry (enquiryId, propertyId, userId, message, createdAt); SavedProperty (userId, propertyId). One owner may create many properties; one property may receive many enquiries; users may save multiple properties. The diagram is a simplified conceptual model, not an extracted database schema.

KEY FEATURES

- Property Discovery: Browse rental property information in one interface.
- Search and Filters: Narrow listings using location, rent range, and property characteristics where supported.
- Property Details: Present descriptions, pricing, photos, and amenities.
- Account Management: Registration, login, and profile workflows where implemented.
- Listing Management: Add, update, or remove properties for authorized owners or administrators where implemented.
- Enquiry and Contact: Provide a path for interested renters to contact a listing representative where implemented.
- Responsive Interface: Support viewing across common desktop and mobile screen sizes.

ROLES AND RESPONSIBILITIES

Tenant / Property Seeker

Searches available properties, compares listing information, reviews details, and submits enquiries or saves preferred listings where these features are available. The user should provide accurate contact information and follow platform guidelines.

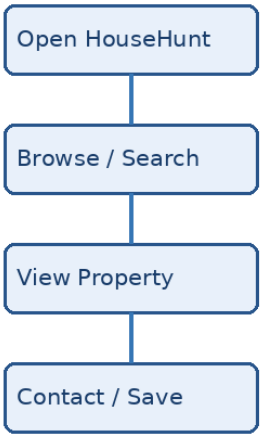
Owner / Property Lister

Creates and maintains accurate property listings, updates availability and pricing, responds to enquiries, and manages owned properties where owner features are implemented.

Administrator

Oversees listing quality, user-related issues, and platform moderation where an administrator module exists. Administrative capabilities must not be assumed to be implemented without checking the application.

USER FLOW



Start: The visitor opens HouseHunt. Browse: The visitor explores listings and uses available filters. View Details: The visitor opens a property and checks rent, location, and amenities. Take Action: The visitor contacts the owner, saves the listing, or returns to search depending on the available interface. End: The visitor closes the site or signs out of the account.

MVC PATTERN

The backend can be organized using the Model–View–Controller (MVC) pattern. Model: Mongoose schemas for user, property, and enquiry data. View: React pages and components render the user-facing interface; API routes expose server operations. Controller: Express controller functions process requests, invoke models, and return responses. This separation improves maintenance, reuse, and testing.

- Model Layer: Database schemas, validation, and data access.
- Controller Layer: Business logic for property search, listing updates, and authentication.
- View / Routing Layer: React UI and Express route definitions.
- Advantages: Modular structure, easier debugging, reusable logic, and simpler feature extension.

PROJECT SETUP AND CONFIGURATION

A standard MERN project can be arranged into client and server directories. The commands below are a reference setup and may differ from the actual HouseHunt repository.

- mkdir househunt && cd househunt
- npm create vite@latest client -- --template react
- cd client && npm install && npm run dev
- cd .. && mkdir server && cd server
- npm init -y
- npm install express mongoose cors dotenv
- node server.js

Configure environment variables for the MongoDB connection string, API port, and client API base URL. Keep secrets in environment files excluded from version control. Deploy the frontend to Netlify and configure the production backend endpoint and CORS settings as needed.

BACKEND DEVELOPMENT

The Node.js/Express backend provides endpoints for application operations. A maintainable reference folder structure is:

- server/config/db.js — database connection
- server/models/User.js — user data model
- server/models/Property.js — rental property model
- server/models/Enquiry.js — enquiry model (optional)
- server/controllers/ — request handling and business logic
- server/routes/ — authentication, property, and enquiry endpoints
- server/middleware/ — authentication and error handling
- server/server.js — Express application entry point

Example API responsibilities include listing properties (GET), retrieving property details (GET), creating a property (POST), updating a property (PUT/PATCH), and deleting a property (DELETE). Route names, authentication methods, and endpoint availability must be confirmed against the actual backend code.

DATABASE DEVELOPMENT

MongoDB is a document-oriented database suitable for storing property information with varying attributes. Mongoose can define schemas, required fields, validation rules, and references between collections. Typical records include users, properties, and enquiries.

Conceptual Property Fields

- title and description: Human-readable listing information.
- location: City, neighborhood, and address information.
- monthlyRent: Numeric rental price.
- propertyType: Apartment, independent house, room, or other category.
- amenities and images: Lists describing facilities and property media.
- ownerId and availability: Listing ownership and status.

Recommended controls include server-side validation, indexes for frequently searched fields, restricted update permissions, and hashed passwords for any locally stored credentials. These are recommended design practices, not verified implementation claims.

FRONT-END DEVELOPMENT

React provides a component-based interface for displaying listings and collecting user input. The following is an illustrative frontend structure; exact file names depend on the completed project.

- src/components/Navbar.jsx — navigation
- src/components/PropertyCard.jsx — listing summary
- src/pages/Home.jsx — landing page
- src/pages/Properties.jsx — browsing and filters
- src/pages/PropertyDetails.jsx — detailed listing
- src/pages/Login.jsx and Signup.jsx — authentication forms, if available
- src/pages/Dashboard.jsx — listing management, if available
- src/services/api.js — API request helpers
- src/App.jsx — routing and application layout

The frontend should support clear navigation, accessible form labels, responsive layouts, loading/error states, and informative empty results when searches return no matches.

OUTPUT SCREENSHOTS

The supplied reference report includes screenshots of the landing page, signup, login, user home, and key feature pages. Because the deployed HouseHunt site could not be accessed from this environment, screenshots have not been fabricated. Replace the spaces below with real screenshots captured from your completed application.

1. Landing / Home Page

[Insert actual screenshot from the deployed HouseHunt application here]

2. Property Listings and Search

[Insert actual screenshot from the deployed HouseHunt application here]

3. Property Details

[Insert actual screenshot from the deployed HouseHunt application here]

4. Login / Signup (if implemented)

[Insert actual screenshot from the deployed HouseHunt application here]

5. Owner / Admin Dashboard (if implemented)

[Insert actual screenshot from the deployed HouseHunt application here]

PROJECT LINKS

Live Project: https://househunt-smartbridge.netlify.app/

Source Code Repository: [Add your HouseHunt GitHub repository URL]

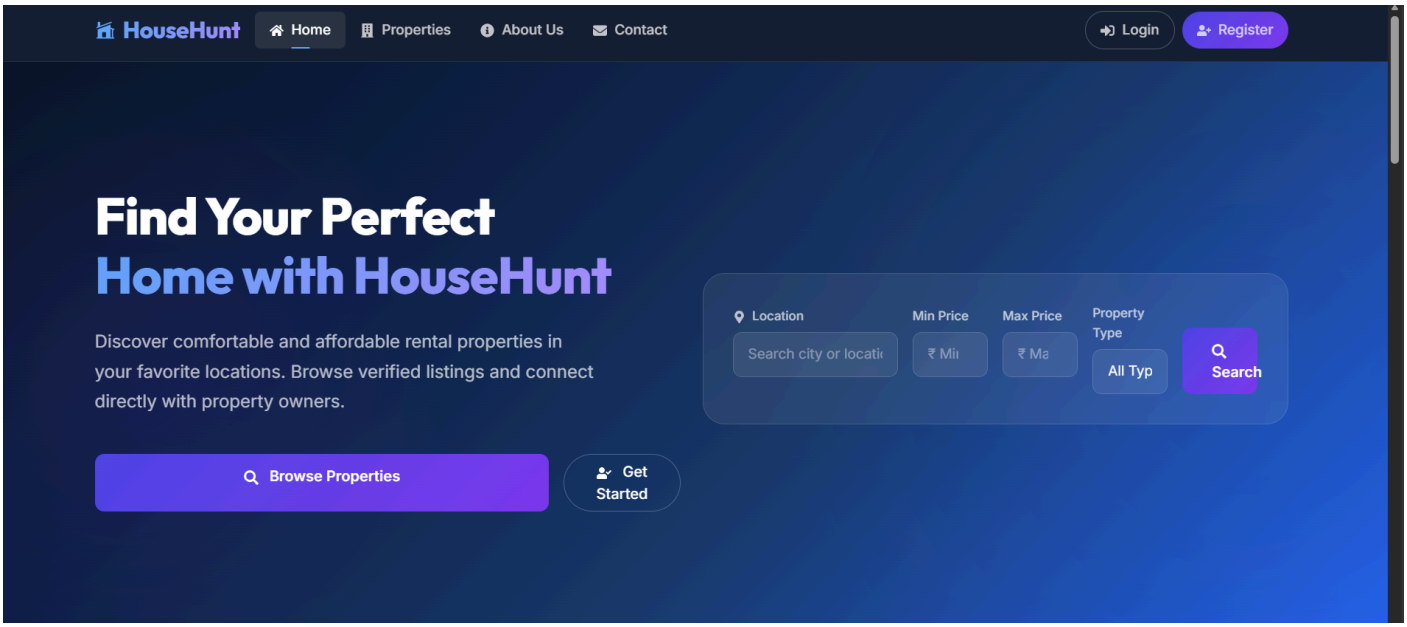
Demo Video: [Add a screen-recording link if required by SmartBridge]

CONCLUSION

HouseHunt presents a practical approach to digital house-rental discovery by organizing property information and simplifying the initial search process. As a MERN full-stack internship project, it demonstrates the relationship between a responsive React interface, REST-based backend services, and document-oriented data management. Future enhancements may include verified property badges, map-based discovery, appointment scheduling, messaging, and recommendation features.

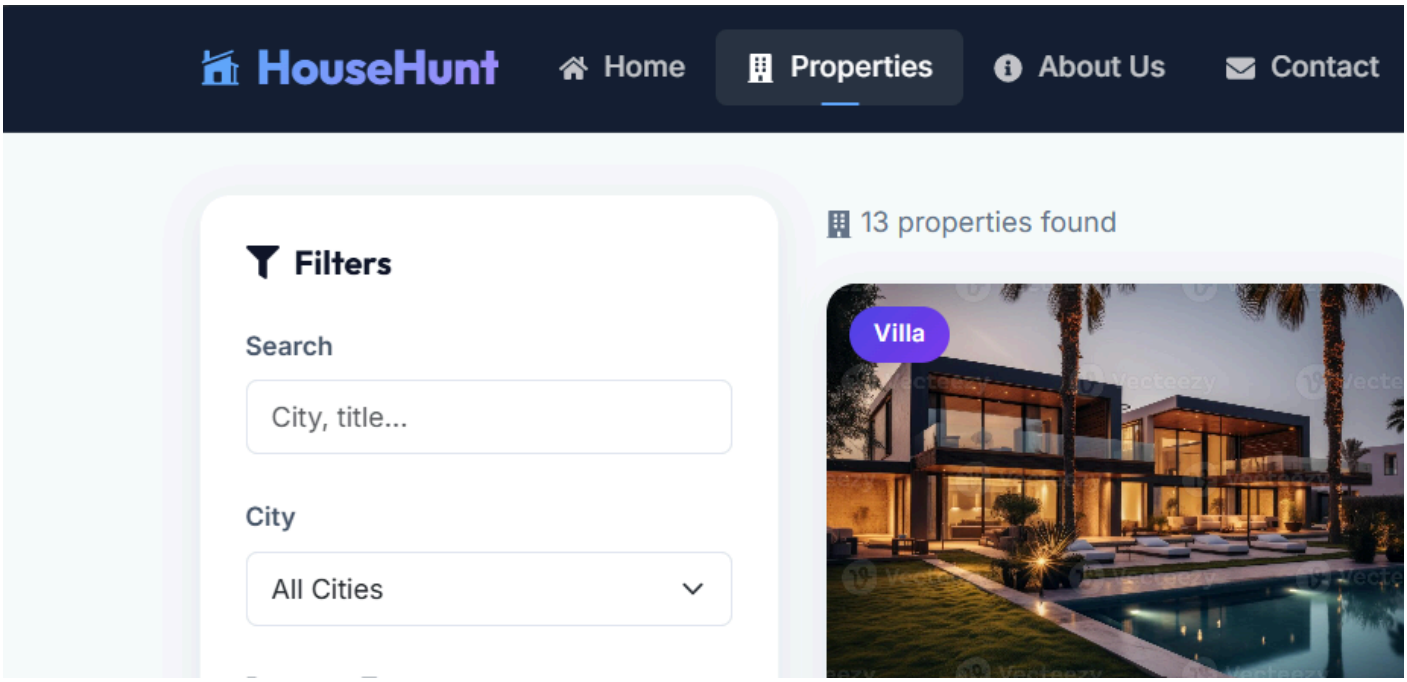
PROJECT OUTPUT SCREENSHOTS

Home Page · Property Search



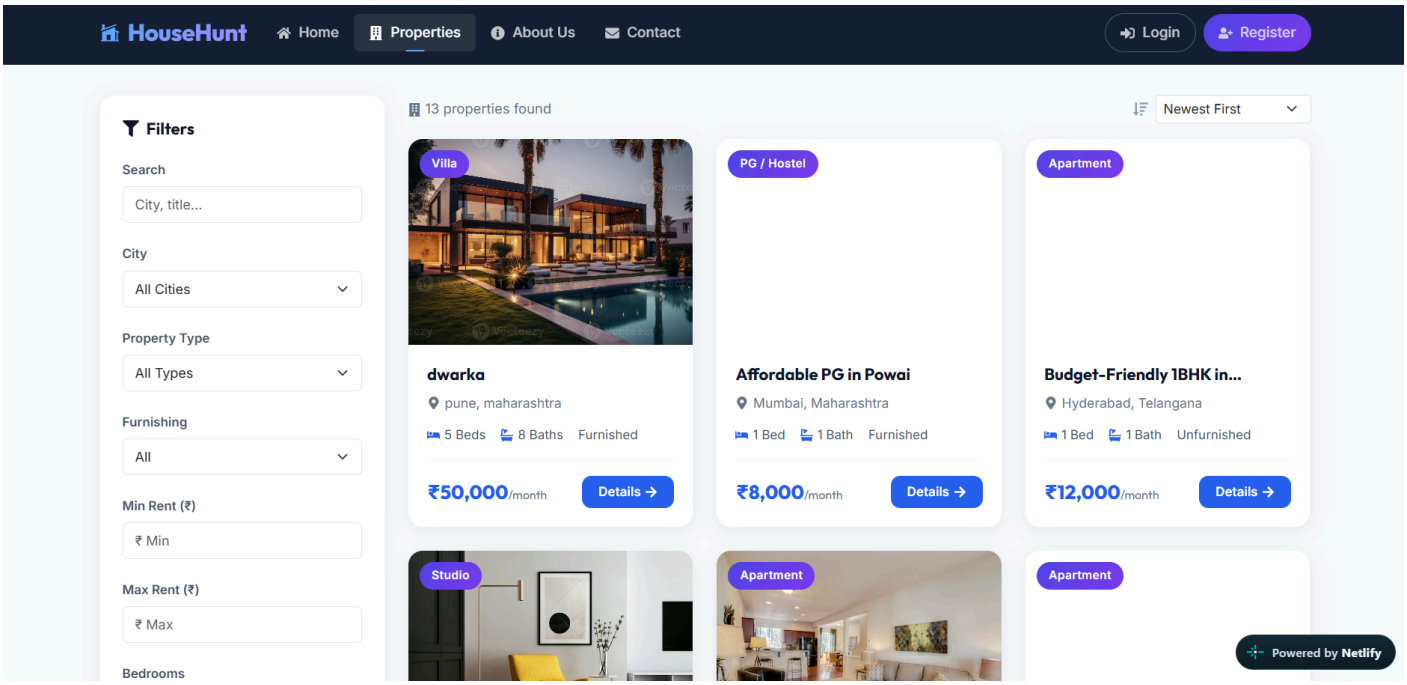
PROJECT OUTPUT SCREENSHOTS

Property Listings · Filter Panel



PROJECT OUTPUT SCREENSHOTS

Property Listings · Search Results



PROJECT OUTPUT SCREENSHOTS

User Registration Page

